

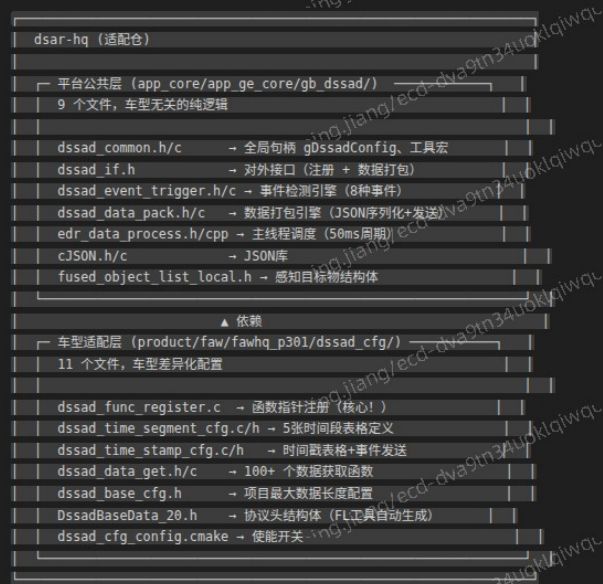
DSSAD 架构与数据链路分析

一、背景：什么是 DSSAD

DSSAD (Data Storage System for Automated Driving) 是 GB/T 国标要求的自动驾驶数据存储系统，功能类似传统汽车的 EDR (事件数据记录器)，但专门面向 L3+ 自动驾驶场景。核心职责是：

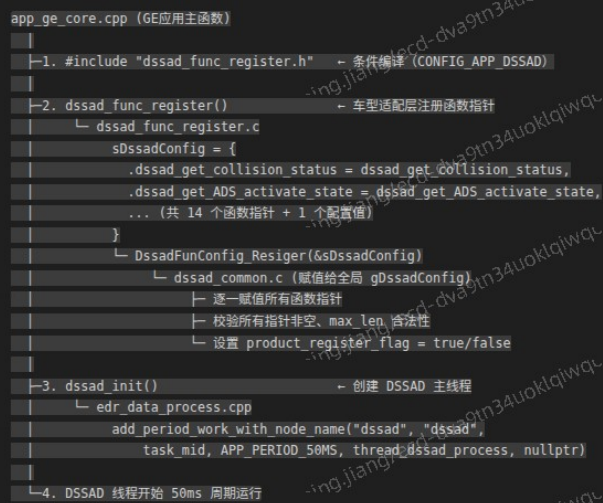
- 1. 时间戳事件记录：在 8 种关键事件触发时，记录“表格1”车辆基本信息 + 协议头，发送给数据挖掘平台 (Fleet Learning)
- 2. 时间段数据记录：以固定周期持续采集 5 张表格的车况数据 (车辆基本/动态/ADS运行/环境感知/驾驶员操作)，以 JSON 格式通过 DCMS topic 分发

二、代码架构：平台层 + 车型适配层

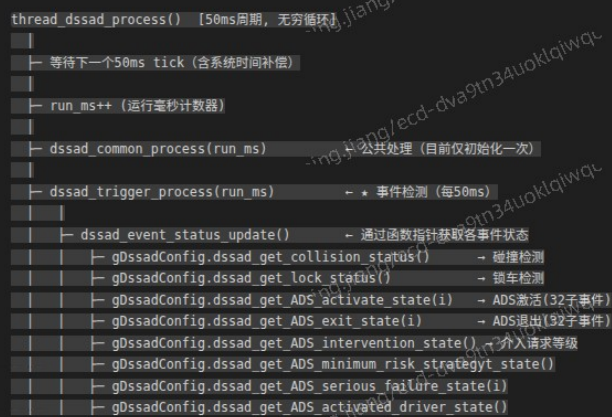


关键设计模式：平台层定义函数指针接口，车型层通过 **gDssadConfig** 全局句柄注册具体实现。

三、初始化链路



四、运行时调度链路 (核心!)



```
└─ gDssadConfig.dssad_get_ADS_Abs_exit_driver_state()
└─ dssad_time_segment_trigger_status_report()  - 时间段事件状态上报
    通过 DCMS_TOPIC_DSSAD_EVENT_CHECK 发送 collision/lock/collision_risk
└─ dssad_time_stamp_event_trigger_check()  - 时间戳事件检测 (边沿触发)
    └─ time_stamp_event_ADS_Auto_Activate_check()  - false-true触发
    └─ time_stamp_event_ADS_Auto_Exit_check()  - false-true触发
    └─ time_stamp_event_InterventionRequest_check()  - 等级递增触发 (0-1-2-3)
    └─ time_stamp_event_MinimumRiskStrategy_check()  - false-true触发
    └─ time_stamp_event_ADS_SeriousFailure_check()  - 逐个DTC上报
    └─ time_stamp_event_ADS_ActivateByDriver_check()  - false-true触发
    └─ time_stamp_event_ADS_ExitByDriver_check()  - false-true触发
└─ 【触发时调用】 gDssadConfig.dssad_time_stamp_data_send(event_id, ...)
    └─ dssad_time_stamp_data_send() in dssad_time_stamp_cfg.c
        └─ 填入 Event Id
        └─ memcpy 到 DssadBaseData (含1578字节meta_data协议头)
        └─ dcms_mcu_topic_send(DCMS_TOPIC_FL_OPT_SEND, ...)
            - 发送给 Fleet Learning / 数据挖掘平台
└─ gDssadConfig.dssad_data_get_process(run_ms)  - 车型数据获取 (在项目层实现)
    从 SOME/IP CAN/LIN 信号中读取最新值, 缓存到全局变量
└─ dssad_data_pack_process(run_ms)  - 时间段数据打包发送
└─ 【每50ms】 gDssadConfig.dssad_time_segment_data_update(time)
    └─ dssad_time_segment_data_update() in dssad_time_segment_cfg.c
        for each of 5 tables:
            if (time % table.report_cycle == 0):
                for each member:
                    调用 member.data_get_function() 从缓存读取最新值
                    dssad_time_segment_data_pack(cur_table, ...)
                        - cJSON 序列化为 JSON 字符串
                        - dcms_mcu_topic_send_msg(topic_id, json_str)
└─ 【每500ms】 gDssadConfig.dssad_time_stamp_data_update()
    └─ dssad_time_stamp_data_update() in dssad_time_stamp_cfg.c
        更新 dssad_time_stamp_data 结构体 (表格1数据的二进制缓存)
```

五、五种数据表格与上报周期

表格	名称	内容	上报周期	DCMS Topic
表格1	veh_base_data	VIN/HW/SW/SN/版本/时间/GPS/里程/故障码 (18项)	1000ms	DCMS_TOPIC_DSSAD_BASE_DATA_PACK
表格2	veh_dynamic_data	车速/有效位/加速度XY/横摆角速度/侧倾角速度/偏航角 (7项)	100ms	DCMS_TOPIC_DSSAD_DYNAMIC_DATA_PACK
表格3	ads_runtime_data	ADS请求: 档位/横纵向加速度/方向盘角度/转向曲率/车速/踏板/灯光/雨刮 (26项)	100ms	DCMS_TOPIC_DSSAD_ADAS_REQ_PACK
表格4	driving_env_data	9个感知目标物: 类型/相对位置XY/相对速度XY/前后距离 (54项)	100ms	DCMS_TOPIC_DSSAD_FUSED_OBJECT_PACK
表格5	driver_handle_data	驾驶员状态: 脱手/安全带/座椅/油门踏板/刹车踏板/方向盘角度/扭矩 (8项)	100ms	DCMS_TOPIC_DSSAD_DRIVER_HANDLE_PACK

表格1同时用于两种场景:

- 时间段数据: 每1秒打包为 JSON 通过 topic 发布
- 时间戳数据: 在8种关键事件触发时, 转为二进制格式 + 协议头 (1578字节元数据), 通过 DCMS_TOPIC_FL_OPT_SEND 直接发送给 Fleet Learning 平台

六、八种时间戳事件

EventId	枚举名	含义	子事件机制	触发条件
0x01	ADS_Activate	自动驾驶激活	32个子事件 (NOD/TJA/ACC/ILCA/AEB/ESA/HPA/NRP/APA/TBA/MOD/MEB/Remote/NOCD...)	false → true 上升沿
0x02	ADS_Exit	自动驾驶退出	32个子事件 (同上对应的退出类型)	false → true 上升沿 (首次默认输出)
0x03	InterventionRequest	介入请求	无, 用等级值(0-1-2-3)	等级递增
0x04	MinimumRiskStrategy	最小风险策略	无	false → true 上升沿
0x05	ADS_SeriousFailure	严重失效	64个DTC故障码	每次上报一个DTC (轮询)
0x06	Collision	碰撞/锁车	无 (时间段事件)	状态变化时写日志
0x07	RiskOfCollision	碰撞风险	无 (时间段事件)	状态变化时写日志
0x08	ADS_UsedByDriver	驾驶员手动操作ADS	2种子事件 (ActivatedByDriver / ExitByDriver)	false → true 上升沿

七、依赖的数据源

车型适配层 `dssad_data_get.c` 中的 100+ 个 `Dssad_Get_XXX()` 函数从以下数据源读取:

数据来源
1. DCMS Topic 订阅
- DCMS_TOPIC_DSSAD_FACTORY_PARAM (工厂参数)
- DCMS_TOPIC_DSSAD_GNSS_INFO (GPS)
- DCMS_TOPIC_DSSAD_EVENT_CHECK (触发状态)
- DCMS_TOPIC_DSSAD_CHASSIS_DYNAMIC (底盘动态)
- DCMS_TOPIC_DSSAD_FUSED_OBJECT (感知目标物)
- DCMS_TOPIC_DSSAD_DMM_ADS_STATUS (ADS状态)
- DCMS_TOPIC_DSSAD_SOMEIP_DIAG (SOME/IP诊断)
2. CAN / SOME/IP 信号 (通过 x_dom_can_rt 代理)
- 车速、里程、加速度、方向盘角度等
- 门锁、前盖、尾门、胎压等车身信号
- 灯光、雨刮等执行器请求
3. DMM (诊断内存管理) 相关
- ADS事件状态 (从MCM侧同步)

- ADS事件数据（OXCMS同步）
- ADS运行时数据（灯光/雨刮请求等）
4. 感知融合目标列表（FusedObjectList）
- 9个目标物的类型/位置/速度

八、核心设计模式总结

- 函数指针注册模式：平台层定义 `DssadConfig_t` (14 个函数指针 + 1 个配置值)，车型层通过 `dssad_func_register()` 绑定具体实现，通过全局 `gDssadConfig` 解耦
- 运行时安全校验：`DssadFunConfig_Resiger()` 注册时会校验所有函数指针不为空，不通过则 `product_register_flag = false`，DSSAD 功能不生效
- 双轨数据输出：
 - 时间段数据（5张表）：cJSON 序列化 → DCMS Topic（通常给数据挖掘/云端消费）
 - 时间戳数据（表格1）：二进制结构体 + 1578字节 FL 协议头 → `DCMS_TOPIC_FL_OPT_SEND`（给 Fleet Learning 平台做事件快照）
- 边沿触发机制：所有时间戳事件都使用 `status_last` vs `status` 的 `false → true` 上升沿触发，避免重复上报
- 调试注入机制：通过 `DCMS_DSSAD_TRIGGER_DEBUG` topic 可以手动注入事件（`dssad_event_debug_flag + dssad_event_debug_data[]`），方便测试验证
- 车型差异化隔离：每个车型有自己独立的 `dssad_cfg/` 目录，`product_build_marcos` CMake 宏在编译时自动 `include` 对应车型的配置，编译进 `app_ge_dssad_${product_name}` 静态库